



Descripción de la Persistencia

Proyecto: Innovatech Solutions

Asignatura: DSY1106 · Desarrollo Full Stack

Autor: Najeeb Escobar Pérez

Fecha: Junio 2026

1. Enfoque general

La persistencia se implementa con **Spring Data JPA** (Hibernate como proveedor) sobre **PostgreSQL 16**. Cada microservicio de negocio accede solo a su propia base mediante interfaces de repositorio; no se usan procedimientos almacenados: la lógica de consulta vive en el código (consultas derivadas del nombre del método o anotadas con `@Query`). El esquema no lo crea Hibernate por reflexión, sino **Flyway** con migraciones versionadas, y Hibernate queda en modo `validate` para garantizar que el código y la base calzan.

2. Database per Service

Cada microservicio es dueño de su esquema; ningún servicio entra a la base de otro. Hay cuatro bases PostgreSQL independientes:

Base	Microservicio	Tablas
projects	ms-projects	projects, tasks
resources	ms-resources	resources
analytics	ms-analytics	kpi_snapshots
keycloak	Keycloak (IdP)	tablas internas de Keycloak

La autonomía de cada servicio se gana a cambio de no tener claves foráneas entre bases: la consistencia entre servicios es eventual y se resuelve con referencias por id y comunicación REST.

3. Entidades y mapeo JPA

Las entidades se mapean con anotaciones JPA. Ejemplo (ms-resources):

```
@Entity
@Table(name = "resources")
public class Resource {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)           private String name;
    @Column(nullable = false, unique = true) private String email;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)           private ResourceRole role;    // DEV, PM, QA...

    @Column(name = "weekly_hours", nullable = false) private Integer weeklyHours;
    @Column(nullable = false)           private Boolean active;
}
```

Los estados de proyectos y tareas se guardan como enumeraciones (`@Enumerated(EnumType.STRING)`): un proyecto está en `PLANNING` / `IN_PROGRESS` / `COMPLETED` / `CANCELLED` y una tarea en `TODO` / `IN_PROGRESS` / `DONE` / `BLOCKED`.

4. Repositorios (patrón Repository)

El acceso a datos se hace con interfaces que extienden `JpaRepository`; Spring Data genera la implementación. Los métodos derivan la consulta de su nombre o la declaran con `@Query`:

```
public interface ResourceRepository extends JpaRepository<Resource, Long> {
    boolean existsByEmail(String email); // valida duplicados en el registro
    Optional<Resource> findByEmail(String email); // resuelve usuario -> recurso
}

public interface KpiSnapshotRepository extends JpaRepository<KpiSnapshot, Long> {
    Optional<KpiSnapshot> findTopByOrderByCapturedAtDesc(); // último snapshot
    List<KpiSnapshot> findByCapturedAtBetweenOrderByCapturedAtAsc( // serie temporal
        Instant from, Instant to);
}
```

5. Relaciones: física vs lógica

Hay dos tipos de relación, y la distinción es clave en una arquitectura database-per-service:

- **Relación física** (clave foránea real), solo dentro de una misma base: una `Task` referencia a su `Project` con `project_id`. Ambas viven en la base `projects`.
- **Relación lógica** (referencia por id, sin FK), entre bases distintas: `Task.assignee_resource_id` apunta a un `Resource` de otra base, y `Project.owner_id` guarda el identificador del usuario de Keycloak. No hay clave foránea; la integridad se valida en la aplicación.

`ms-analytics` no lee las bases de los otros: calcula los KPIs llamando por REST a `ms-projects` y `ms-resources`, y guarda en su propia base los snapshots periódicos.

6. Esquema versionado con Flyway

Las tablas se crean con migraciones SQL numeradas en `src/main/resources/db/migration`. Flyway las aplica en orden al arrancar y registra cuáles ya corrieron, de modo que el esquema es reproducible en cualquier entorno. Ejemplo (`ms-projects`):

```
-- V001__schema_projects.sql
CREATE TABLE projects (
    id                BIGSERIAL PRIMARY KEY,
    name              VARCHAR(200) NOT NULL,
    description        VARCHAR(1000),
    status             VARCHAR(30)  NOT NULL,
    start_date         DATE,
    planned_end_date   DATE,
    owner_id           VARCHAR(64)    -- referencia lógica a Keycloak
);

-- V003__schema_tasks.sql
CREATE TABLE tasks (
    id                BIGSERIAL PRIMARY KEY,
    project_id         BIGINT NOT NULL REFERENCES projects (id), -- FK física
    title              VARCHAR(200) NOT NULL,
    status             VARCHAR(30)  NOT NULL,
    assignee_resource_id BIGINT      -- referencia lógica a resources
);
```

Hibernate corre con `ddl-auto: validate`: no modifica el esquema, solo verifica que las entidades calcen con las tablas que creó Flyway; si no calzan, el servicio no arranca.

7. Persistencia en el despliegue

En Kubernetes, cada PostgreSQL corre como un **StatefulSet** con un volumen persistente propio (`volumeClaimTemplates`, un PVC por réplica). Es el tipo de objeto adecuado para una carga con estado: identidad de red estable y datos que sobreviven al reinicio del pod.